Arrays — Exercícios

Estruturas de Dados Básicas e Complexidade

Exercícios com Respostas

Material Didático

Exercícios Teóricos

Exercício 1 — Rastreando uma Sequência de Operações

Considere um `ArrayDinamico` com capacidade inicial 4 e tamanho 0. A seguinte sequência de operações é executada:

`inserirNoFinal(10)`, `inserirNoFinal(20)`, `inserirNoInicio(5)`, `inserirNoFinal(30)`, `inserirNoFinal(40)`, `inserirNoFinal(50)`.

Lembre-se de que, quando o array fica cheio, a estratégia de redimensionamento dobra a capacidade.

a) Desenhe o estado do array (conteúdo, tamanho e capacidade) após **cada** operação.

b) Para cada operação, indique quantos deslocamentos ou cópias de elementos foram necessários (inserção no início desloca elementos; redimensionamento copia todos).

c) Em qual operação ocorreu o redimensionamento? Qual era a capacidade antes e depois?

d) Qual foi o custo total (soma de todos os deslocamentos e cópias) da sequência inteira?

Resposta:

a) Estado após cada operação (capacidade inicial = 4):

#	Operação	Array	Tam/Cap
1	`inserirNoFinal(10)`	$[10, _, _, _]$	1/4
2	`inserirNoFinal(20)`	$[10, 20, _, _]$	2/4
3	`inserirNoInicio(5)`	$[5, 10, 20, _]$	3/4
4	`inserirNoFinal(30)`	$[5, 10, 20, 30]$	4/4
5	`inserirNoFinal(40)`	$[5, 10, 20, 30, 40, _, _, _]$	5/8
6	`inserirNoFinal(50)`	$[5, 10, 20, 30, 40, 50, _, _]$	6/8

b) Deslocamentos/cópias por operação: (1) 0; (2) 0; (3) 2 deslocamentos (10 e 20 para a direita); (4) 0; (5) 4 cópias de redimensionamento + 1 inserção = 5; (6) 0.

c) O redimensionamento ocorreu na operação 5. Capacidade antes: 4; depois: 8.

d) Custo total: $0 + 0 + 2 + 0 + 5 + 0 = 7$ operações elementares (além das 6 escritas de inserção propriamente ditas).

Exercício 2 — Encontrando o Erro

Um programador implementou o seguinte método na classe `ArrayDinamico` para remover **todas** as ocorrências de um valor v:

Código 1: Método com erro

```
public void removerTodos(int v) {
    for (int i = 0; i < tamanho; i++) {
        if (dados[i] == v) {
```

```
            removerNaPosicao(i);
        }
    }
}
```

Considere que `removerNaPosicao(i)` desloca os elementos seguintes para a esquerda e decrementa `tamanho`.

a) Teste mentalmente o método para $A = [3, 7, 3, 3, 5]$ com $v = 3$. Mostre o estado do array e o valor de i a cada iteração do laço. O método funciona corretamente?

b) Identifique o erro e explique por que ele ocorre.

c) Proponha uma correção e reescreva o método corrigido.

Resposta:

a) Rastreamento para $A = [3, 7, 3, 3, 5]$, $v = 3$:

$i = 0$: $A[0] = 3 = v$, remove posição 0 $\rightarrow A = [7, 3, 3, 5]$, tamanho=4.

$i = 1$: $A[1] = 3 = v$, remove posição 1 $\rightarrow A = [7, 3, 5]$, tamanho=3.

$i = 2$: $A[2] = 5 \neq v$, nada.

$i = 3$: $3 \geq 3$, sai do laço. Resultado: $[7, 3, 5]$.

O método **não funciona**: sobrou um 3 na posição 1.

b) O erro ocorre porque, ao remover a posição i, o próximo elemento ocupa a posição i, mas o laço incrementa i e pula esse elemento sem examiná-lo.

c) Correção: decrementar i após a remoção para reexaminar a mesma posição:

Código 2: Método corrigido

```
public void removerTodos(int v) {
    for (int i = 0; i < tamanho; i++) {
        if (dados[i] == v) {
            removerNaPosicao(i);
            i--;
        }
    }
}
```

Exercício 3 — Endereçamento em Memória

Um array de inteiros A de tamanho 8 começa no endereço de memória 500. Cada `int` ocupa 4 bytes.

a) Calcule o endereço de memória de $A[0]$, $A[3]$ e $A[7]$.

b) Um programador acessa $A[8]$ por engano. Qual endereço de memória seria lido? Esse endereço pertence ao array?

c) Suponha que imediatamente após A na memória exista uma variável `int x` (no endereço 532). Se o programador executar `A[8] = 99`, o que acontece com o valor de `x`? Como se chama esse tipo de erro?

Resposta:

a) endereço$(A[i]) = 500 + i \times 4$.

$A[0] = 500, \quad A[3] = 500 + 12 = 512, \quad A[7] = 500 + 28 = 528.$

b) $A[8]$: endereço $500 + 32 = 532$. Esse endereço **não pertence** ao array (que vai de 500 a 531).

c) Como `x` está no endereço 532, a escrita `A[8] = 99` sobrescreve o valor de `x` com 99. Esse tipo de erro é chamado de **buffer overflow** (estouro de buffer) — uma escrita fora dos limites do array que corrompe dados adjacentes na memória.

Exercício 4 — Contando Comparações

Considere o seguinte método que verifica se um array contém algum valor duplicado:

Código 3: Verificação de duplicatas

```
public static boolean temDuplicata(int[] A) {
    for (int i = 0; i < A.length; i++) {
        for (int j = i + 1; j < A.length; j++) {
            if (A[i] == A[j]) return true;
        }
    }
    return false;
}
```

a) Para $A = [4, 1, 7, 1, 9]$, em qual par (i, j) o método retorna `true`? Quantas comparações (`A[i] == A[j]`) são feitas até esse ponto?

b) Para $A = [4, 1, 7, 3, 9]$ (sem duplicatas), quantas comparações são feitas no total?

c) Generalize: para um array de tamanho n sem duplicatas, quantas comparações são feitas? Qual é a complexidade assintótica de pior caso?

Resposta:

a) Para $A = [4, 1, 7, 1, 9]$:

$i = 0$: compara $(0, 1)$, $(0, 2)$, $(0, 3)$, $(0, 4)$ — 4 comparações, nenhum match.

$i = 1$: compara $(1, 2)$, $(1, 3)$ — na comparação $(1, 3)$: $A[1] = 1 = A[3] = 1 \rightarrow$ retorna `true`.

Total: $4 + 2 = 6$ comparações. Par: $(i, j) = (1, 3)$.

b) Para $A = [4, 1, 7, 3, 9]$ $(n = 5)$: $i = 0$: 4 comp.; $i = 1$: 3 comp.; $i = 2$: 2 comp.; $i = 3$: 1 comp. Total: $4 + 3 + 2 + 1 = 10$.

Professor Rodrigo Bossini
⟨professorbossini@gmail.com⟩
https://professorbossini.dev

c) Para tamanho n: $(n-1)+(n-2)+\cdots+1 = \dfrac{n(n-1)}{2}$ comparações. Complexidade: $\Theta(n^2)$.

Exercício 5 — Quando o Redimensionamento Importa

Considere dois cenários para inserir $n = 1000$ elementos no final de um `ArrayDinamico` com capacidade inicial 1:

- **Estratégia A:** quando o array fica cheio, a capacidade é **dobrada**.
- **Estratégia B:** quando o array fica cheio, a capacidade é **aumentada em 1**.

a) Na Estratégia A, quantas vezes ocorre redimensionamento para inserir 1000 elementos? Quais são as capacidades após cada redimensionamento?

b) Na Estratégia A, qual é o número total aproximado de cópias causadas pelos redimensionamentos? (Dica: é a soma $1 + 2 + 4 + 8 + \cdots$)

c) Na Estratégia B, quantas vezes ocorre redimensionamento? Qual é o número total de cópias? (Dica: é a soma $1 + 2 + 3 + \cdots + 999$)

d) Compare os dois totais. Qual estratégia é mais eficiente? Em termos de custo médio por inserção, qual é a ordem de grandeza de cada uma?

Resposta:

a) Redimensionamentos na Estratégia A: capacidades $1 \to 2 \to 4 \to 8 \to 16 \to 32 \to 64 \to 128 \to 256 \to 512 \to 1024$. São 10 redimensionamentos.

b) Cópias: $1 + 2 + 4 + 8 + 16 + 32 + 64 + 128 + 256 + 512 = 1023$. Aproximadamente $2n$.

c) Na Estratégia B: 999 redimensionamentos (a cada inserção a partir da 2ª). Cópias: $1 + 2 + 3 + \cdots + 999 = \dfrac{999 \times 1000}{2} = 499\,500$.

d) Estratégia A: $\sim$1023 cópias para 1000 inserções $\to$ custo médio $\approx$ 1 cópia por inserção $\to \Theta(1)$ amortizado. Estratégia B: $\sim$500 000 cópias $\to$ custo médio $\approx$ 500 por inserção $\to \Theta(n)$ amortizado. A estratégia de dobrar é muito mais eficiente.

Exercícios Práticos

Exercício 6 — Remover Todas as Ocorrências

Com base no problema identificado no Exercício 2, implemente em Java um método correto `removerTodos(int v)` na classe `ArrayDinamico` que remove **todas** as ocorrências do valor v. O método deve funcionar mesmo quando há valores repetidos consecutivos.

Inclua um `main` que teste com $A = [3, 7, 3, 3, 5, 3, 8]$ removendo $v = 3$ e imprima o resultado esperado $[7, 5, 8]$.

Resposta:

Código 4: removerTodos correto

```java
// Dentro da classe ArrayDinamico
public void removerTodos(int v) {
    for (int i = 0; i < tamanho; i++) {
        if (dados[i] == v) {
            removerNaPosicao(i);
            i--;
        }
    }
}

public static void main(String[] args) {
    ArrayDinamico arr = new ArrayDinamico(10);
    for (int v : new int[]{3, 7, 3, 3, 5, 3, 8})
        arr.inserirNoFinal(v);
    System.out.println("Antes: " + arr);
    arr.removerTodos(3);
    System.out.println("Depois: " + arr);
    // Saida: Depois: [7, 5, 8]
}
```

Exercício 7 — Compactar Zeros

Implemente em Java um método `compactarZeros()` na classe `ArrayDinamico` que move todos os zeros para o final do array, mantendo a ordem relativa dos demais elementos. O método deve operar in-place (sem criar um novo array).

Por exemplo, $[0, 3, 0, 0, 5, 7, 0, 2]$ deve se tornar $[3, 5, 7, 2, 0, 0, 0, 0]$.

Dica: use um índice auxiliar que marca a próxima posição onde um valor diferente de zero deve ser colocado. Percorra o array e, sempre que encontrar um valor diferente de zero, coloque-o na posição marcada e avance o índice.

Inclua um `main` para teste. Qual é a complexidade do seu método?

Resposta:

Código 5: Compactar zeros

```java
// Dentro da classe ArrayDinamico
public void compactarZeros() {
    int pos = 0;
    for (int i = 0; i < tamanho; i++) {
        if (dados[i] != 0) {
            dados[pos] = dados[i];
            pos++;
        }
    }
    while (pos < tamanho) {
        dados[pos] = 0;
        pos++;
    }
}

public static void main(String[] args) {
    ArrayDinamico arr = new ArrayDinamico(10);
    for (int v : new int[]{0, 3, 0, 0, 5, 7, 0, 2})
        arr.inserirNoFinal(v);
    System.out.println("Antes: " + arr);
    arr.compactarZeros();
    System.out.println("Depois: " + arr);
    // Saida: Depois: [3, 5, 7, 2, 0, 0, 0, 0]
}
```

Complexidade: $\Theta(n)$ — duas passagens lineares.

Exercício 8 — Intercalação de Dois Arrays Ordenados

Implemente em Java um método estático `intercalar(int[] A, int[] B)` que recebe dois arrays **já ordenados** em ordem crescente e retorna um **novo** array contendo todos os elementos de ambos, também em ordem crescente. O método deve operar em $\Theta(n+m)$, onde n e m são os tamanhos de A e B: use dois índices, um para cada array, e a cada passo copie o menor elemento para o array resultado.

Inclua um `main` que teste com $A = [1, 5, 8, 12]$ e $B = [2, 3, 9, 20]$, imprimindo o resultado esperado $[1, 2, 3, 5, 8, 9, 12, 20]$.

Resposta:

Código 6: Intercalação de arrays ordenados

```java
public class Intercalacao {

    public static int[] intercalar(int[] A, int[] B) {
        int[] resultado = new int[A.length + B.length];
        int i = 0, j = 0, k = 0;
        while (i < A.length && j < B.length) {
            if (A[i] <= B[j]) {
                resultado[k] = A[i];
                i++;
            } else {
                resultado[k] = B[j];
                j++;
            }
            k++;
        }
        while (i < A.length) {
            resultado[k] = A[i];
            i++;
            k++;
        }
        while (j < B.length) {
            resultado[k] = B[j];
            j++;
            k++;
        }
        return resultado;
    }

    public static void main(String[] args) {
        int[] A = {1, 5, 8, 12};
        int[] B = {2, 3, 9, 20};
        int[] R = intercalar(A, B);
        System.out.print("[");
        for (int x = 0; x < R.length; x++) {
            System.out.print(R[x]);
            if (x < R.length - 1) System.out.print(", ");
        }
        System.out.println("]");
        // Saida: [1, 2, 3, 5, 8, 9, 12, 20]
    }
}
```

Exercício 9 — Histograma de Notas

Implemente em Java uma classe `Histograma` que recebe um array de inteiros (notas de 0 a 100) e exibe a quantidade de notas em cada faixa de 10 pontos: $[0, 10)$, $[10, 20)$, ..., $[90, 100]$.

Use um array de 10 posições como tabela de contagem. A faixa de cada nota deve ser calculada por divisão inteira (sem `if` encadeados). A nota 100 deve ser tratada como pertencente à última faixa $[90, 100]$.

O método `exibir()` deve imprimir cada faixa com sua contagem e uma barra visual feita de caracteres `#`. Exemplo: `[70, 80):  5 #####`.

Inclua um `main` que teste com o array $[85, 42, 73, 91, 68, 55, 100, 37, 78, 62, 90, 44, 71, 88, 59]$.

Resposta:

Código 7: Histograma de notas

```java
public class Histograma {

    public static void exibir(int[] notas) {
        int[] contagem = new int[10];
        for (int nota : notas) {
            int faixa = nota / 10;
            if (faixa >= 10) faixa = 9;
            contagem[faixa]++;
        }
        for (int f = 0; f < 10; f++) {
            int inicio = f * 10;
            int fim = (f == 9) ? 100 : inicio + 10;
            String intervalo = (f == 9)
                ? "[" + inicio + ", " + fim + "]"
                : "[" + inicio + ", " + fim + ")";
            StringBuilder barra = new StringBuilder();
            for (int j = 0; j < contagem[f]; j++)
                barra.append("#");
            System.out.printf("%-12s %d %s%n",
                intervalo, contagem[f], barra);
        }
    }

    public static void main(String[] args) {
        int[] notas = {85, 42, 73, 91, 68, 55, 100,
                       37, 78, 62, 90, 44, 71, 88, 59};
        exibir(notas);
    }
}
```

Exercício 10 — Rotação Circular

Implemente em Java um método `rotacionar(int k)` na classe `ArrayDinamico` que desloca todos os elementos k posições para a direita de forma circular: o elemento que "sai" pela direita volta pela esquerda.

Por exemplo, $[1, 2, 3, 4, 5]$ com $k = 2$ resulta em $[4, 5, 1, 2, 3]$.

É permitido usar um array auxiliar de tamanho n. O método deve funcionar para qualquer $k \geq 0$ (inclusive $k > n$).

Dica: para $k > n$, observe que rotacionar n posições retorna ao array original. Portanto, basta considerar $k \bmod n$.

Inclua um `main` que teste com $k = 2$ e com $k = 7$ para um array de 5 elementos.

Resposta:

Código 8: Rotação circular

```java
// Dentro da classe ArrayDinamico
public void rotacionar(int k) {
    if (tamanho == 0) return;
    k = k % tamanho;
    if (k == 0) return;
    int[] aux = new int[tamanho];
    for (int i = 0; i < tamanho; i++) {
        aux[(i + k) % tamanho] = dados[i];
    }
    for (int i = 0; i < tamanho; i++) {
        dados[i] = aux[i];
    }
}

public static void main(String[] args) {
    ArrayDinamico arr = new ArrayDinamico(10);
    for (int v : new int[]{1, 2, 3, 4, 5})
        arr.inserirNoFinal(v);

    System.out.println("Original: " + arr);
    arr.rotacionar(2);
    System.out.println("k=2: " + arr);
    // Saida: [4, 5, 1, 2, 3]

    ArrayDinamico arr2 = new ArrayDinamico(10);
    for (int v : new int[]{1, 2, 3, 4, 5})
        arr2.inserirNoFinal(v);
    arr2.rotacionar(7); // 7 % 5 = 2
    System.out.println("k=7: " + arr2);
    // Saida: [4, 5, 1, 2, 3]
}
```

Referências

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. *Introduction to Algorithms*. 4th edition. MIT Press, 2022. Capítulo 10: Elementary Data Structures.

Professor Rodrigo Bossini
⟨professorbossini@gmail.com⟩
https://professorbossini.dev